Summary Report: Control Transfer Statements

(`break`, `continue`, and `pass`)

This report summarises the functionality, syntax, and constraints of control transfer statements in Python, drawing from the provided session material.

---

1. Introduction to Control Transfer Statements

Control transfer statements, also known as control flow statements, are essential tools in Python programming. They allow programmers to **manage and alter the flow of execution** within loops (`for` and `while`) or conditional statements (`if`).

Without these statements, programming tasks would be "less flexible and more cumbersome". Using them makes code **more concise, readable, and maintainable** by clearly expressing intentions for handling various flow scenarios.

The three main control transfer statements in Python are:

1. **`break`**: Used to prematurely exit a loop.
2. **`continue`**: Used to skip the current iteration and proceed to the next one.
3. **`pass`**: Used to maintain syntactic structure without taking any action.

---

2. The `break` Statement

The `break` statement is used to **immediately terminate** the enclosing loop (`for` or `while`) where it is encountered.

Core Functionality

- When executed, the control flow jumps **out of the loop** to the next statement following the loop structure.
- It is often condition-based, meaning the loop exits once a specific condition is met, regardless of whether the loop's natural termination condition has been reached (i.e., whether the loop has iterated through all elements or the condition has evaluated to `False`).

Syntax and Example Usage

The `break` statement is primarily used within `while` loops or `for` loops.

Context	Example Code	Execution/Output
For Loop	`for i in range(5): print(i); if i == 2: break`	The loop exits after printing 2.

While Loop	`i = 1; while i < 6: print(i); if i == 3: break; i += 1`	The loop prints 1, 2, 3 and then terminates prematurely.
Infinite Loop	`while True: count += 1; if count == 3: break`	The loop executes exactly three times before the `break` statement exits it.
Search Function	`for value in my_list: if value == 30: print("Found!"); break`	If 30 is found, it prints "Value found!" and exits, skipping the remaining list elements.

Constraints and Notes on `break`

- **Location Constraint:** The `break` statement is **only allowed inside loops** (`for` and `while`). Attempting to use it outside a loop (e.g., directly inside an `if` block not nested in a loop) results in a `SyntaxError: 'break' outside loop`.
- **List Comprehensions:** `break` is **not allowed** within list comprehensions or generator expressions and will result in a `SyntaxError`.
- **Nested Loops:** When used in nested loops, `break` only exits the **innermost loop** where it is encountered; the outer loops continue their execution unaffected.
- **`else` Block:** The `else` block associated with a `for` or `while` loop is **NOT executed** if the loop is terminated prematurely by a `break` statement.

---

3. The `continue` Statement

The `continue` statement is used to **skip the remaining statements** in the current iteration of a loop and move the control flow **immediately to the next iteration**. The statement allows the loop to continue running without being terminated.

Core Functionality

- It is used when a specific condition is met that renders the rest of the code block in the current iteration irrelevant.
- After executing `continue`, the program skips straight to checking the loop condition for the next iteration.

Syntax and Example Usage

Context	Example Code	Execution/Output
For Loop	`for num in numbers: if num % 2 == 0: continue; print(num)`	Prints only odd numbers (`1, 3, 5, 7, 9`). When an even number is encountered, `print(num)` is skipped.
While Loop	`while num <= 10: if num % 2 != 0: num += 1; continue; print(num); num += 1`	Prints only even numbers (`2, 4, 6, 8, 10`).
Nested Loop	`while j < 5: j += 1; if j == 3: continue; print(j)`	The value 3 is skipped in the output because `continue` prevents the `print(j)` line from executing when `j` is 3.

Constraints and Notes on `continue`

- **Loop Requirement:** Similar to `break`, `continue` must be used within a loop context. Placing it improperly, especially outside a loop or within a conditional block that is not fully enclosed in a loop, can result in a `SyntaxError`.

---

4. The `pass` Statement

The `pass` statement is a **null operation** used as a placeholder. It is executed when a statement is syntactically required, but the programmer does not wish to implement any code or command at that point.

Core Functionality

- It performs **no action**.
- Its primary role is to maintain the syntactic validity of a block of code (e.g., in an `if` statement or a loop body) when the logic is yet to be implemented.

Syntax and Example Usage

Context	Example Code	Execution/Output
While Loop	`while count < 5: pass; print(count); count += 1`	The `pass` statement acts as a placeholder inside the loop body.
For Loop	`for fruit in fruits: pass`	The loop iterates through the list, but no action is taken inside the body.
Conditional (`if`)	`if b > a: pass; print("End of program")`	If the condition is true, `pass` ensures the syntax is valid, and the program proceeds to print "End of program".

Notes on `pass`

- **Syntactic Necessity:** `pass` prevents an `IndentationError` that would occur if a code block (like an `if` body or loop body) were left completely empty.